Pipeline descriptions

Contents

- Vector arithmetic logic unit (VALU)
- Scalar arithmetic logic unit (SALU)
- Local data share (LDS)
- Branch
- Scheduler
- Matrix fused multiply-add (MFMA)
- Non-pipeline resources

This section details the various execution pipelines of the compute unit.

Vector arithmetic logic unit (VALU)

The vector arithmetic logic unit (VALU) executes vector instructions over an entire wavefront, each work-item (or, vector-lane) potentially operating on distinct data. The VALU of a CDNA-CDNA4 architecture-based GPUs or GCN™ GPU typically consists of:

- Four 16-wide SIMD processors (see Introduction to AMD GPU Programming with HIP (slide 24) for more details).
- Four 64 or 128 KiB VGPR files (yielding a total of 256-512 KiB total per CU), see AGPRs for more detail.
- An instruction buffer (per-SIMD) that contains execution slot 32 total wavefront slots on each CU).
- A vector memory (VMEM) unit which transfers data between VGPRs and memory; each

docs-10.1.0

work-item supplies its own memory address and supplies or receives unique data.

- CDNA-CDNA4 architecture-based GPUs (such as the MI100 and MI2XX), contain additional Matrix Fused Multiply-Add (MFMA) units.

To support branching and conditionals, each wavefront in the VALU has a distinct execution mask which determines which work-items in the wavefront are active for the currently executing instruction. When executing a VALU instruction, inactive work-items (according to the current execution mask of the wavefront) do not execute the instruction and are treated as no-ops.

> **Note**
>
> On GCN GPUs and the CDNA architecture-based AMD Instinct MI100 GPUs, there are slots for up to 10 wavefronts in the instruction buffer, but generally occupancy is limited by other factors to 32 waves per compute unit. On the CDNA2 architecture-based AMD Instinct MI2XX Series GPUs, there are only 8 waveslots per-SIMD.

Scalar arithmetic logic unit (SALU)

The scalar arithmetic logic unit (SALU) executes instructions that are shared between all work-items in a wavefront. This includes control flow such as if/else conditionals, branches and looping pointer arithmetic, loading common values, and more.

The SALU consists of:

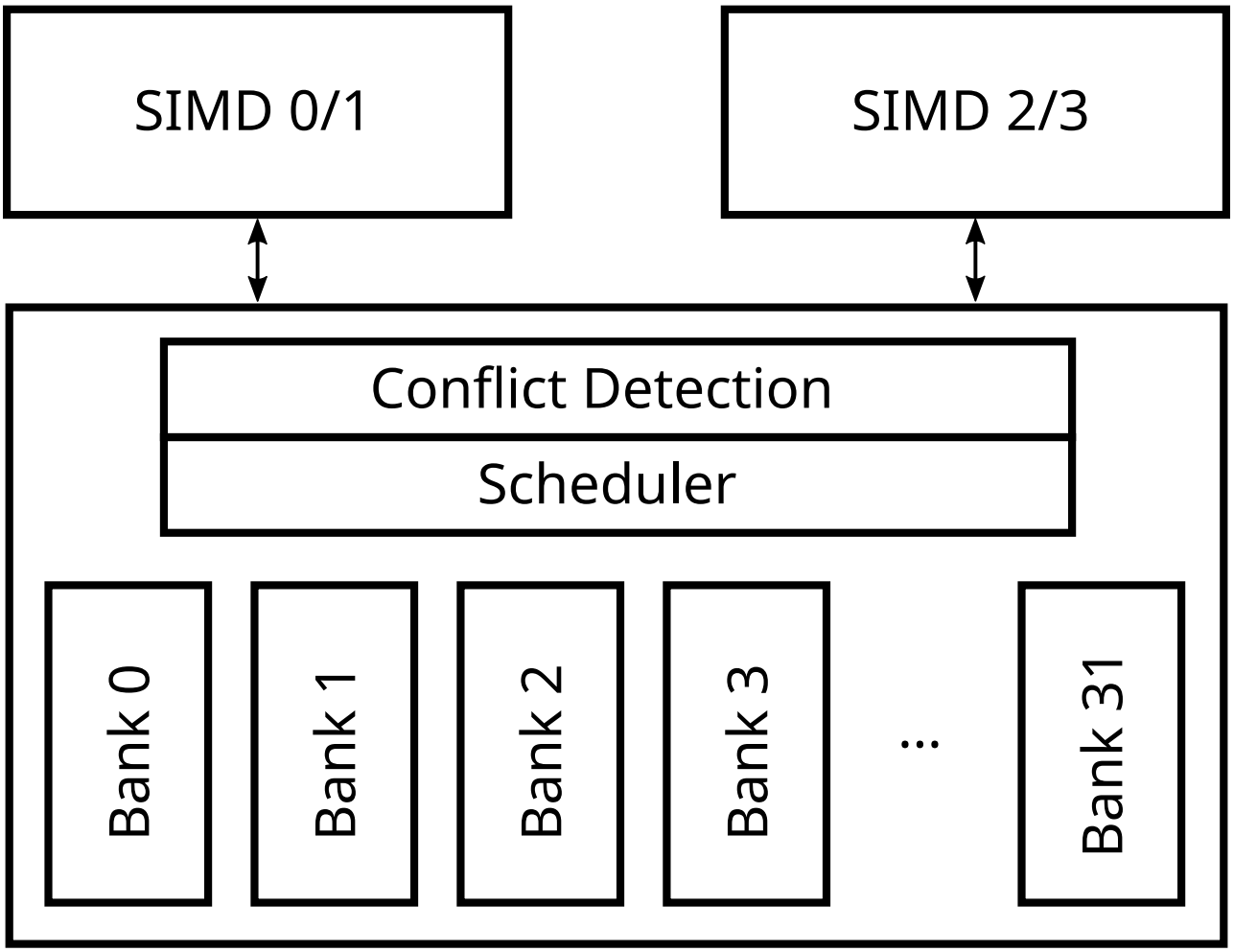
- A scalar processor capable of various arithmetic, conditional, and comparison (etc.) operations. See Chapter 5. Scalar ALU Operations of the CDNA2 Instruction Set Architecture (ISA) Reference Guide for more detail.
- A 12.5 KiB Scalar General Purpose Register (SGPR) file
- A scalar memory (SMEM) unit which transfers data between SGPRs and memory

Data loaded by the SMEM can be cached in the scalar L1 data cache, and is typically only used for read-only, uniform accesses such as kernel arguments, memory.

docs-10.1.0

Local data share (LDS)

The local data share (LDS, a.k.a., “shared memory”) is fast on-CU scratchpad that can be explicitly managed by software to effectively share data and to coordinate between wavefronts in a workgroup.



Performance model of the local data share (LDS) on AMD Instinct MI-series GPUs.

This image is the performance model of the LDS on CDNA architecture-based GPUs (adapted from AMD Radeon HD7970 with GCN Architecture (slide 20)). The SIMDs in the VALU are connected to the LDS in pairs (see above). Only one SIMD per pair may issue an LDS instruction at a time, but both pairs may issue concurrently.

On CDNA architecture-based GPUs, the LDS contains 32 banks and each bank is 4B wide. The LDS is designed such that each bank can be read from, writ docs-10.1.0 updated every cycle, for a total throughput of 128B/clock (The AMD GCN Architecture - A Crash Course (slide 40)).

On each of the two ports to the SIMDs, 64B can be sent in each direction per cycle. So, a single wavefront, coming from one of the 2 SIMDs in a pair, can only get back 64B/cycle (16 lanes per cycle). The input port is shared between data and address and this can affect achieved bandwidth for different data sizes. For example, a 64-wide store where each lane is sending a 4B value takes 8 cycles (50% peak bandwidth) while a 64-wide store where each lane is sending a 16B value takes 20 cycles (80% peak bandwidth).

In addition, the LDS contains conflict-resolution hardware to detect and handle bank conflicts. A bank conflict occurs when two (or more) work-items in a wavefront want to read, write, or atomically update different addresses that map to the same bank in the same cycle. In this case, the conflict detection hardware will determine a new schedule such that the access is split into multiple cycles with no conflicts in any single cycle.

When multiple work-items want to read from the same address within a bank, the result can be efficiently broadcasted (The AMD GCN Architecture - A Crash Course (slide 41)). Multiple work-items writing to the same address within a bank typically results undefined behavior in HIP and other high-level languages, as the LDS will write the value from the last work-item as determined by the hardware scheduler (The AMD GCN Architecture - A Crash Course (slide 41)). This behavior may be useful in the very specific case of storing a uniform value.

Relatedly, an address conflict is defined as occurring when two (or more) work-items in a wavefront want to atomically update the same address on the same cycle. As in a bank-conflict, this may cause additional cycles of work for the LDS operation to complete.

Branch

The branch unit is responsible for executing jumps and branches to execute control flow operations. Note that Branch operations are not used for execution mask updates, but only for “whole wavefront” control-flow changes.

Scheduler

The scheduler is responsible for arbitration and issue of instructi

docs-10.1.0

currently executing on the CU. On every clock cycle, the scheduler:

- Considers waves from one of the SIMD units for execution, selected in a round-robin fashion between the SIMDs in the compute unit
- Issues up to one instruction per wavefront on the selected SIMD
- Issues up to one instruction per each of the instruction categories among the waves on the selected SIMD:
  - VALU
  - VMEM operations
  - SALU / SMEM operations
  - LDS
  - Branch operations

This gives a maximum of five issued Instructions Per Cycle (IPC), per-SIMD, per-CU (Introduction to AMD GPU Programming with HIP, The AMD GCN Architecture - A Crash Course). On CDNA architecture-based GPUs with MFMA instructions, these are issued via the VALU. Some of them will execute on a separate functional unit and typically allow other VALU operations to execute in their shadow (see the MFMA section for more detail).

> **Note**
>
> The IPC model used by ROCm Compute Profiler omits the following two complications for clarity. First, CDNA architecture-based GPUs contain other execution units on the CU that are unused for compute applications. Second, so-called “internal” instructions (see The AMD GCN Architecture - A Crash Course (slide 29)) are not issued to a functional unit, and can technically cause the maximum IPC to *exceed* 5 instructions per-cycle in special (largely unrealistic) cases. The latter issue is discussed in more detail in the ‘internal’ IPC example.

Matrix fused multiply-add (MFMA)

CDNA architecture-based GPUs, such as the MI100 and MI2XX, contain specialized hardware to accelerate matrix-matrix multiplications, also known as Matrix Fused Multiply-Add (MFMA) operations. The exact operation types and supporte[illegible] different GPUs. For a general discussion of these hardware units[illegible] blog post on GPUOpen . In addition, to explore the available MFMA instructions in-depth on

docs-10.1.0

various AMD GPUs (including the CDNA line), see the AMD Matrix Instruction Calculator:

Partial snapshot of the AMD Matrix Instruction Calculator Tool

```
$ ./matrix_calculator.py –architecture cdna2 –instruction v_mfma_f32_4x4x1f
Architecture: CDNA2
Instruction: V_MFMA_F32_4X4X1F32
    Encoding: VOP3P-MAI
    VOP3P Opcode: 0x42
    VOP3P-MAI Opcode: 0x2
    Matrix Dimensions:
        M: 4
        N: 4
        K: 1
        blocks: 16
    Execution statistics:
        FLOPs: 512
        Execution cycles: 8
        FLOPs/CU/cycle: 256
        Can co-execute with VALU: True
        VALU co-execution cycles possible: 4
    Register usage:
        GPRs required for A: 1
        GPRs required for B: 1
        GPRs required for C: 4
        GPRs required for D: 4
        GPR alignment requirement: 8 bytes
```

For the purposes of ROCm Compute Profiler, the MFMA unit is typically treated as a separate pipeline from the VALU, as other VALU instructions (along with other execution pipelines such as the SALU) typically can be issued during a portion of the total duration of an MFMA operation.

Note

The exact details of VALU and MFMA operation co-execution vary by instruction, and can be explored in more detail via the following fields in the AMD Matrix Instruction Calculator's detailed instruction information:

- `Can co-execute with VALU`
- `VALU co-execution cycles possible`

docs-10.1.0

Non-pipeline resources

In this section, we describe a few resources that are not standalone pipelines but are important for understanding performance optimization on CDNA architecture-based GPUs.

Barrier

Barriers are resources on the compute-unit of a CDNA architecture-based GPU that are used to implement synchronization primitives (for example, HIP's `__syncthreads`). Barriers are allocated to any workgroup that consists of more than a single wavefront.

Accumulation vector general-purpose registers (AGPRs)

Accumulation vector general-purpose registers, or AGPRs, are special resources that are accessible to a subset of instructions focused on MFMA operations. These registers allow the MFMA unit to access more than the normal maximum of 256 architected vector general-purpose registers (VGPRs) by having up to 256 in the architected space and up to 256 in the accumulation space. Traditional VALU instructions can only use VGPRs in the architected space, and data can be moved to/from VGPRs↔AGPRs using specialized instructions (`v_accvgpr_*`). These data movement instructions may be used by the compiler to implement lower-cost register-spill/fills on architectures with AGPRs.

AGPRs are not available on all AMD Instinct GPUs. The AMD Instinct MI100 (CDNA) has a 2x256 KiB register file, where one half is available as general-purpose VGPRs, and the other half is for matrix math accumulation VGPRs (AGPRs). The AMD Instinct MI2XX (CDNA2) has a 512 KiB VGPR file per CU, where each wave can dynamically request up to 256 KiB of VGPRs and an additional 256 KiB of AGPRs. For more information, refer to this comment.

Terms and Conditions

ROCm Licenses and Disclaimers

Privacy

Trademarks

Supply Chain Transparency

Fair and Open Competition

docs-10.1.0

UK Tax Strategy

Cookie Policy

Cookie Settings

© 2026 Advanced Micro Devices, Inc